Part 2 — Alpha–Beta Pruning: Explanation and Python Code

Let's first understand Alpha–Beta Pruning using your existing game tree from Part 1. Then we will complete the missing `alpha_beta_decision()` function and calculate the number of terminal leaves evaluated and cutoff events.

1. What is Alpha–Beta Pruning?

Alpha–Beta Pruning is an optimization of the Minimax algorithm that avoids evaluating branches that cannot affect the final decision.

In Part 1, Minimax evaluated all 12 terminal leaves. Alpha–Beta Pruning tries to avoid some of this work while producing the same optimal decision.

It uses two values:

- **Alpha (α):** The best value MAX can guarantee so far.
- **Beta (β):** The best value MIN can guarantee so far.

The rule is:

$$
\boxed{\alpha \geq \beta \implies \text{Prune}}
$$

Pruning means stopping the evaluation of the remaining children of the current node because they cannot improve the final decision.

Simple example

Suppose MAX has two choices, A and B.

- MAX evaluates A and discovers that its minimax value is 5.
- Therefore, MAX can already guarantee a value of at least 5. So, (\alpha=5).
- Now MAX evaluates B, where MIN is choosing between two options.
- The first option under B has value 3.

MIN can choose that option, so B can have a value of at most 3.

Because MAX already has an alternative worth 5, MAX will never choose B if B's value cannot exceed 3.

Therefore:

$$
\alpha=5,\qquad \beta=3
$$

Since:

$$
5 \geq 3
$$

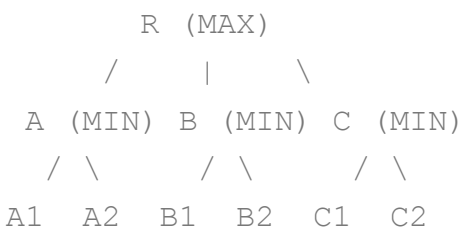
we can prune the remaining children of B.

Important: We do not need to calculate the remaining values under B because they cannot change MAX's decision at the root.

---

2. Understanding Alpha and Beta in your game tree

Your tree has this structure:



Each node at the third level is a MAX node, and its children contain terminal utility values.

The children must be evaluated in their existing left-to-right order. We must not rearrange them to make pruning more effective.

Let's walk through the tree.

Step 1: Evaluate A

Initially:

$$
\alpha=-\infty,\qquad \beta=+\infty
$$

At A, MIN evaluates A1 first.

A1 contains terminal values 3 and 5.

Since A1 is a MAX node:

$$
A1=\max(3,5)=5
$$

MIN now knows that its current best value at A is 5.

Next, A evaluates A2, which contains 6 and 9.

$$
A2=\max(6,9)=9
$$

MIN chooses the smaller of A1 and A2:

$$
A=\min(5,9)=5
$$

The root R is a MAX node, so it updates alpha:

$$
\alpha=\max(-\infty,5)=5
$$

The best option found so far is A, with a value of 5.

Step 2: Evaluate B

Now R evaluates B.

Because B is a MIN node, its initial beta value is $+\infty$.

B evaluates B1 first:

$$
B1=\max(1,2)=2
$$

B updates beta:

$$
\beta=\min(+\infty,2)=2
$$

At this point:

$$
\alpha=5,\qquad \beta=2
$$

Since:

$$
\alpha\geq\beta
$$

we have:

$$
5\geq2
$$

A cutoff occurs at B.

B's remaining child, B2, is pruned. We do not evaluate its terminal leaves 0 and -1.

This is our first pruning event.

The root's current best value remains 5.

Step 3: Evaluate C

Now R evaluates C.

C is a MIN node, and the root's current alpha is still 5.

C evaluates C1 first:

$$
C1=\max(7,4)=7
$$

C updates beta:

$$
\beta=\min(+\infty,7)=7
$$

Since (5<7), pruning cannot happen yet.

C evaluates C2:

$$
C2=\max(5,6)=6
$$

C then calculates:

$$
C=\min(7,6)=6
$$

At the root, MAX compares the values it has obtained.

$$
R=\max(5,6)=6
$$

There is no cutoff at C because its beta value never becomes less than or equal to the root's alpha value.

Final result

The resulting minimax values are:

Node	Minimax value
A	5
B	0
C	6
R	6

Notice that B's exact value is 0, but Alpha–Beta Pruning does not need to calculate that exact value to determine the root's optimal decision.

It only needs to know that B cannot beat A's value of 5.

---

3. Implement Alpha–Beta Pruning

Your starter code already contains the `alpha_beta()` function. You only need to complete `alpha_beta_decision()`.

However, let's understand what the existing function does.

Understanding `alpha_beta()`

The function returns two things:

```
return value, stats
```

- `value` : The minimax value calculated for the current node.
- `stats` : A dictionary that counts the terminal leaves evaluated and the cutoff events.

For example:

```
stats = {'leaves': 0, 'prunes': 0}
```

Whenever the function reaches a terminal node, it increases the leaf counter:

```
stats['leaves'] += 1
```

Whenever the pruning condition is satisfied, it increases the cutoff counter:

```
stats['prunes'] += 1
```

The function then stops evaluating the remaining children of that node using `break`.

Understanding `alpha_beta_decision()`

This function must:

1. Start with alpha equal to negative infinity and beta equal to positive infinity.
2. Evaluate the root's children in their existing order.
3. Select the child with the highest value.
4. Update alpha whenever MAX finds a better value.
5. Return the best child, its value, and the statistics.

We must pass the same statistics dictionary through the recursive calls so that all evaluated leaves and cutoff events are counted together.

4. Complete Python code

Run the following code in your notebook. It includes the game tree, terminal utilities, Alpha–Beta algorithm, and decision function.

import math

GAME_TREE = {
'R': ['A', 'B', 'C'],
'A': ['A1', 'A2'],
'B': ['B1', 'B2'],
'C': ['C1', 'C2'],
'A1': ['A1a', 'A1b'],
'A2': ['A2a', 'A2b'],
'B1': ['B1a', 'B1b'],
'B2': ['B2a', 'B2b'],
'C1': ['C1a', 'C1b'],
'C2': ['C2a', 'C2b']
}

UTILITY = {
'A1a': 3, 'A1b': 5,
'A2a': 6, 'A2b': 9,
'B1a': 1, 'B1b': 2,
'B2a': 0, 'B2b': -1,
'C1a': 7, 'C1b': 4,
'C2a': 5, 'C2b': 6
}

def alpha_beta(node, alpha=-math.inf, beta=math.inf,
maximizing_player=True, stats=None):

```
if stats is None:
    stats = {'leaves': 0, 'prunes': 0}

if node in UTILITY:
    stats['leaves'] += 1
    return UTILITY[node], stats

children = GAME_TREE[node]

if maximizing_player:
    value = -math.inf
```

```
    for child in children:
        child_value, stats = alpha_beta(
            child, alpha, beta, False, stats
        )

        value = max(value, child_value)
        alpha = max(alpha, value)

        if alpha >= beta:
            stats['prunes'] += 1
            break

    return value, stats

value = math.inf

for child in children:
    child_value, stats = alpha_beta(
        child, alpha, beta, True, stats
    )

    value = min(value, child_value)
    beta = min(beta, value)

    if alpha >= beta:
        stats['prunes'] += 1
        break

return value, stats
```

```
def alpha_beta_decision(root='R'):
best_child = None
best_value = -math.inf
alpha = -math.inf
beta = math.inf

stats = {'leaves': 0, 'prunes': 0}

for child in GAME_TREE[root]:
    value, stats = alpha_beta(
        child, alpha, beta, False, stats
```

```
    )

    if value > best_value:
        best_child = child
        best_value = value

    alpha = max(alpha, best_value)

return best_child, best_value, stats
```

```
move, value, stats = alpha_beta_decision('R')

print('Best move :', move)
print('Value :', value)
print('Leaves examined:', stats['leaves'])
print('Prunes :', stats['prunes'])
```

Expected output

```
Best move       : C
Value           : 6
Leaves examined: 9
Prunes          : 2
```

---

5. Which terminal leaves were evaluated?

Because we preserve the left-to-right ordering of `GAME_TREE`, the algorithm evaluates the following terminal leaves.

Order	Terminal leaf	Utility	Result
1	A1a	3	Evaluated
2	A1b	5	Evaluated
3	A2a	6	Evaluated
4	B1a	1	Evaluated
5	B1b	2	Evaluated
6	C1a	7	Evaluated

Order	Terminal leaf	Utility	Result
7	C1b	4	Evaluated
8	C2a	5	Evaluated
9	C2b	6	Evaluated

Total terminal leaves evaluated: 9 out of 12.

Which branches were pruned?

There are two cutoff events.

Cutoff 1: A2

After evaluating A1, MIN at A has a current value of 5.

When evaluating A2, the first terminal leaf, A2a, has a value of 6. This makes the alpha value at that MAX node 6, while its beta bound is 5.

Since:

$$
6\geq5
$$

the algorithm prunes the remaining child, **A2b**, whose utility is 9.

Cutoff 2: B

At the root, MAX already has a value of 5 from A.

When B1 returns a value of 2, MIN at B can already keep its value at 2 or lower. MAX therefore has no reason to choose B over A.

Since:

$$
\alpha=5,\qquad\beta=2
$$

and $5\geq2$, the algorithm prunes the remaining child B2, including both terminal leaves **B2a (0) and B2b (-1)**.

Thus, the pruned terminal leaves are:

- A2b
- B2a
- B2b

Three terminal leaves are skipped, and two cutoff events occur.

---

6. Compare Minimax and Alpha–Beta Pruning

Complete the assignment table as follows:

Measure	Minimax	Alpha–Beta
Optimal move	C	C
Root value	6	6
Terminal leaves evaluated	12	9
Cutoff events	0	2

Alpha–Beta evaluates **3 fewer terminal leaves**, reducing the number of terminal evaluations from 12 to 9.

This represents a reduction of:

$$
\frac{12-9}{12}\times100=25\%
$$

So, for this particular tree and child ordering, Alpha–Beta Pruning reduces terminal-leaf evaluations by **25%**.

Note that the number of cutoff events is not necessarily equal to the number of terminal leaves skipped. One cutoff can skip an entire subtree.

---

7. Explanation to write in your assignment

Alpha–Beta Pruning improves the Minimax algorithm by avoiding the evaluation of branches that cannot affect the final decision. It maintains two values: alpha, which represents the best value MAX can guarantee so far, and beta, which represents the best value MIN can guarantee so far. A cutoff occurs when alpha is greater than or equal to beta.

The algorithm evaluates the children from left to right, preserving the order in GAME_TREE. During the search, it prunes the remaining child A2b because its MAX node reaches a value of 6 while its beta bound is 5. It also prunes the subtree rooted at B2 because MIN at B can achieve a value of at most 2, while MAX already has an alternative worth 5. These are two cutoff events.

Alpha–Beta Pruning evaluates 9 terminal leaves instead of all 12 leaves evaluated by Minimax. Despite skipping some branches, it returns the same optimal first move, C, with a root value of 6. Pruning does not change the optimal decision because the skipped branches cannot improve the outcome for the player making the relevant decision.

Final concept to remember

Minimax finds the best move; Alpha–Beta Pruning finds the same best move while avoiding unnecessary calculations.

For your assignment:

- **Optimal move:** C
- **Root value:** 6
- **Terminal leaves evaluated:** 9
- **Cutoff events:** 2
- **Terminal leaves skipped:** 3